Java

JAVA FUNDAMENTALS

Complete Study Guide with Examples & Flowcharts

Variables | Data Types | Operators | Input / Output

B.Tech – Artificial Intelligence & Machine Learning

Karunya Institute of Technology and Sciences

Batch 2023 – 2027

TABLE OF CONTENTS

01 Introduction to Java

Platform-independent, Object-oriented Programming Language

What is Java?

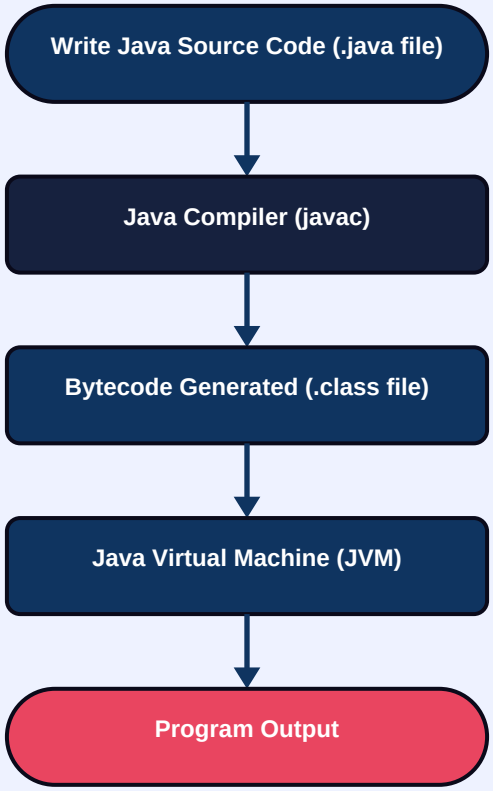
Java is a high-level, object-oriented, platform-independent programming language developed by Sun Microsystems (later acquired by Oracle). Its 'Write Once, Run Anywhere' (WORA) principle means code compiled once runs on any device with a Java Virtual Machine (JVM).

Key Features of Java

Feature	Description
Platform Independent	Code runs on any OS via JVM
Object-Oriented	Everything is modelled as objects and classes
Secure	Built-in security features and bytecode verification
Robust	Strong memory management and exception handling
Multithreaded	Supports concurrent execution of multiple threads

Java Program Execution Flow

Java Execution Flow



The four pillars covered in this guide form the foundation of every Java program:

Variables	Named memory containers for data
Data Types	Defines what kind of data a variable holds
Operators	Symbols to compute, compare, and assign values
I/O Operations	Receive input from users and display output

02 Variables in Java

Named memory locations to store and manipulate data

What is a Variable?

A variable is a named memory location that stores data during program execution. Think of it as a labelled storage box — the label is the variable name, and the contents are the stored value. The box type (data type) determines what can be stored.

Three Components of a Variable

Component	Example	Description
Data Type	int	Specifies what kind of value is stored
Variable Name	age	Identifier used to access the stored value
Value	20	The actual data assigned to the variable

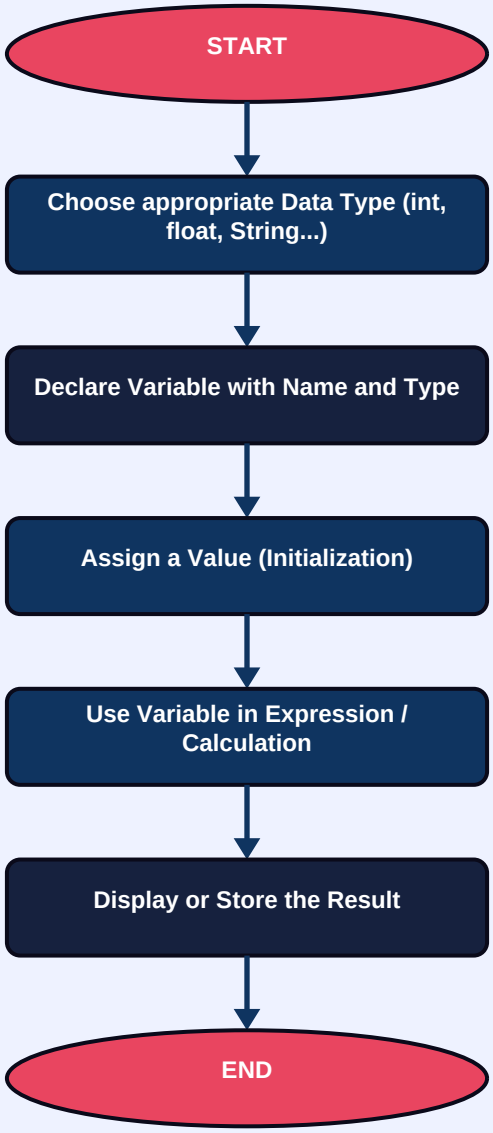
Declaration and Initialization

Variable Declaration Syntax

```
// 1. Declaration Only
int age;

// 2. Declaration + Initialization
int age = 20;

// 3. Assign Value Later
int age;
age = 20;

// Multiple variable examples
float salary = 35000.50f;
char grade = 'A';
String name = "John";
boolean isPassed = true;
```

Variable Declaration & Usage Flow

Variable Declaration & Usage Flowchart



Naming Rules & Conventions

■ VALID Names	■ INVALID Names	■ Best Practices
int age	int 1age (starts with digit)	Start with lowercase letter
double totalMarks	float student marks (spaces)	Use camelCase for multi-words
String studentName	double @salary (special char)	Use meaningful, descriptive names
float salaryAmount	int class (reserved keyword)	Avoid single-letter names

Example Program – Variables in Action

VariableExample.java

```
public class VariableExample {
        public static void main(String[] args) {
                int mark1 = 85;          // integer variable
                int mark2 = 90;          // integer variable
                int total = mark1 + mark2; // derived variable

                System.out.println("Mark 1 = " + mark1);
                System.out.println("Mark 2 = " + mark2);
                System.out.println("Total Marks = " + total);
        }
}

// OUTPUT:
// Mark 1 = 85
// Mark 2 = 90
// Total Marks = 175
```

03 Data Types in Java

Specifies the type of value a variable can hold

What are Data Types?

A data type defines the kind of value a variable can store, the memory allocated, the range of valid values, and the operations that can be performed. Java is strongly typed — every variable must have a declared type before use.

Java Data Type Categories

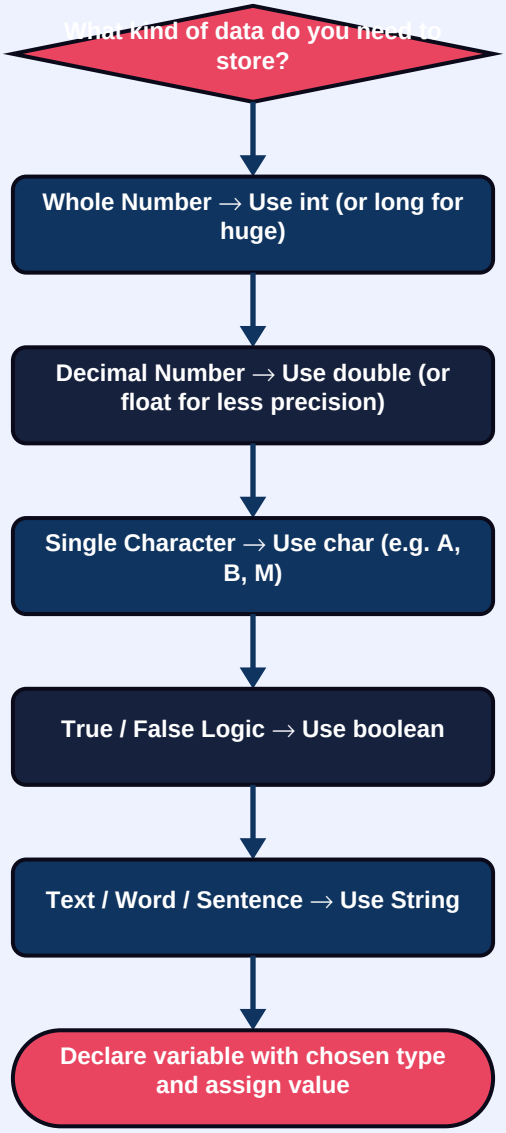
Primitive Data Types (8 built-in types)	**Non-Primitive (Reference) Types** (user-defined / library)
byte, short, int, long float, double, char, boolean	String, Arrays, Classes, Interfaces

Primitive Data Types – Quick Reference

Type	Size	Range / Values	Example	Use Case
`byte`	1 byte	-128 to 127	`byte x = 100;`	Memory saving
`short`	2 bytes	-32,768 to 32,767	`short yr = 2025;`	Small integers
`int`	4 bytes	-2.1B to 2.1B	`int age = 20;`	General counting
`long`	8 bytes	Very large whole nums	`long pop = 1400000000L;`	Large numbers
`float`	4 bytes	~6-7 decimal digits	`float pct = 89.5f;`	Measurements
`double`	8 bytes	~15-16 decimal digits	`double pi = 3.14159;`	Precision math
`char`	2 bytes	Single Unicode char	`char g = 'A';`	Characters
`boolean`	1 bit	true / false only	`boolean b = true;`	Flags / logic

Choosing the Right Data Type – Decision Flow

Data Type Selection Flowchart



Example Program – All Data Types

DataTypeExample.java

```
public class DataTypeExample {
        public static void main(String[] args) {
                int age = 20;
                float percentage = 89.5f;
                double salary = 35000.75;
                char grade = 'A';
                boolean isPassed = true;
                String name = "Rahul";

                System.out.println("Name: "       + name);
                System.out.println("Age: "        + age);
                System.out.println("Percentage: " + percentage);
                System.out.println("Salary: "     + salary);
                System.out.println("Grade: "      + grade);
                System.out.println("Passed: "     + isPassed);
        }
}

// OUTPUT:
// Name: Rahul      Age: 20      Percentage: 89.5
// Salary: 35000.75  Grade: A     Passed: true
```

04 Operators in Java

Special symbols to compute, compare, assign, and evaluate

What are Operators?

Operators are special symbols that perform operations on variables and values (operands). Java provides 5 main operator categories: Arithmetic, Relational, Logical, Assignment, and Increment/Decrement. They are the engine of every computation and decision in code.

1 · Arithmetic Operators

Used to perform mathematical calculations on numeric values.

Operator	Name	Example (a=20, b=10)	Result
+	Addition	a + b	30
-	Subtraction	a - b	10
*	Multiplication	a * b	200
/	Division	a / b	2
%	Modulus (Remainder)	a % b	0

Arithmetic Operators Example

```
int a = 20, b = 10;
System.out.println(a + b);   // 30  – Addition
System.out.println(a - b);   // 10  – Subtraction
System.out.println(a * b);   // 200 – Multiplication
System.out.println(a / b);   // 2   – Division
System.out.println(a % b);   // 0   – Modulus
```

2 · Relational Operators

Compare two values and always return a boolean (true or false).

Operator	Meaning	Example (a=15, b=10)	Result
==	Equal to	a == b	false
!=	Not equal to	a != b	true
>	Greater than	a > b	true
<	Less than	a < b	false
>=	Greater than or equal	a >= b	true
<=	Less than or equal	a <= b	false

3 · Logical Operators

Combine multiple conditions to produce a single true/false result.

Operator	Name	Returns true when...	Example
&&	Logical AND	BOTH conditions are true	(age>=18 && hasID) → true
||	Logical OR	AT LEAST ONE condition is true	(marks>=50 || marks>=40) → true
!	Logical NOT	Condition is FALSE (reverses)	(!isAbsent) → true

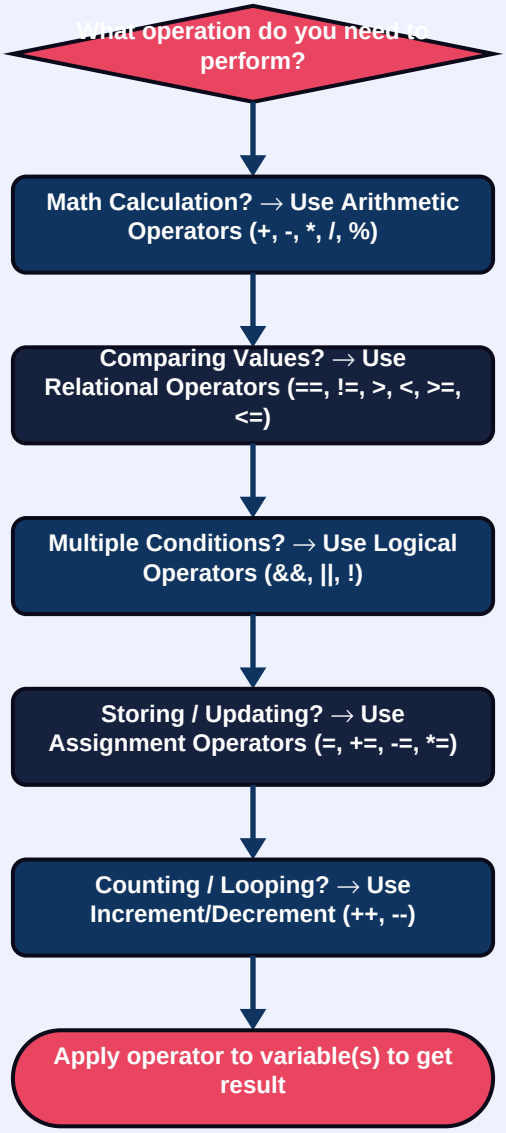
4 · Assignment & 5 · Increment/Decrement Operators

Assignment & Increment/Decrement

```
// Assignment Operators
int a = 10;
a += 5;    // a = a + 5  → 15
a -= 3;    // a = a - 3  → 12
a *= 2;    // a = a * 2  → 24
a /= 4;    // a = a / 4  → 6
a %= 4;    // a = a % 4  → 2

// Increment / Decrement
int count = 5;
count++;   // Post-increment → count = 6
count--;   // Post-decrement → count = 5
++count;   // Pre-increment  → count = 6
--count;   // Pre-decrement  → count = 5
```

Operators Decision Flowchart

Which Operator Should I Use?



05

Input / Output (I/O) in Java

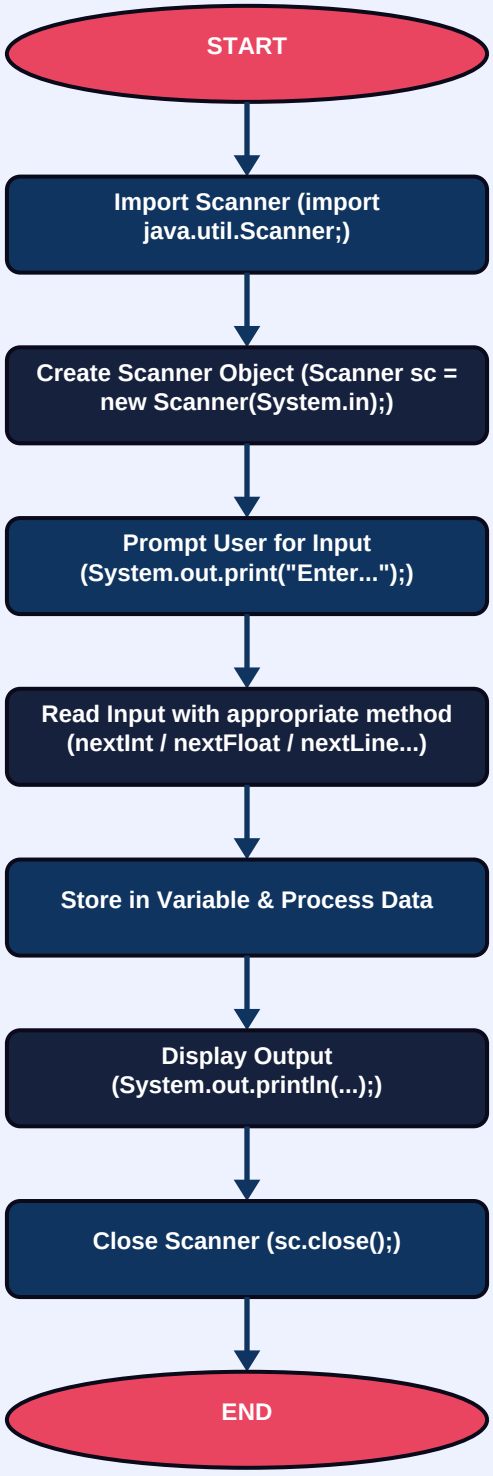
Receiving data from users and displaying processed results

What is I/O in Java?

I/O (Input/Output) enables communication between a program and its users. Input = receiving data (via keyboard using Scanner class). Output = displaying results (using System.out.print / println / printf). Together they make programs interactive and useful.

Input → Processing → Output Cycle

Complete I/O Program Flow



Scanner Class – Input Methods

Method	Reads	Example	Data Type
`sc.nextInt()`	Integer value	`int age = sc.nextInt();`	`int`
`sc.nextFloat()`	Float decimal	`float pct = sc.nextFloat();`	`float`

`sc.nextDouble()`	Double decimal	`double sal = sc.nextDouble();`	**`double`**
`sc.next().charAt(0)`	Single character	`char g = sc.next().charAt(0);`	**`char`**
`sc.nextLine()`	Full line of text	`String nm = sc.nextLine();`	**`String`**
`sc.nextBoolean()`	true or false	`boolean b = sc.nextBoolean();`	**`boolean`**
`sc.nextLong()`	Long integer	`long n = sc.nextLong();`	**`long`**

Output Methods Comparison

Method	Behaviour	Example	Output
`System.out.print()`	No newline at end	`print("Hi"); print(" Java");`	**`Hi Java`**
`System.out.println()`	Adds newline after	`println("Hi"); println("Java");`	**`Hi`** **`Java`**
`System.out.printf()`	Formatted output	`printf("%.2f", 3.14159);`	**`3.14`**

Complete I/O Program Example

StudentInfo.java – Full I/O Example

```
import java.util.Scanner;

public class StudentInfo {
        public static void main(String[] args) {
                Scanner sc = new Scanner(System.in);

                // INPUT
                System.out.print("Enter your name: ");
                String name = sc.nextLine();

                System.out.print("Enter your age: ");
                int age = sc.nextInt();

                System.out.print("Enter your marks: ");
                float marks = sc.nextFloat();

                // PROCESSING
                boolean passed = marks >= 40;
                float total = marks + 10;
                float average = total / 2;

                // OUTPUT
                System.out.println("\n--- Student Details ---");
                System.out.println("Name   : " + name);
                System.out.println("Age    : " + age);
                System.out.println("Marks  : " + marks);
                System.out.println("Passed : " + passed);
                System.out.printf("Average: %.2f%n", average);

                sc.close();
        }
}
```

06

Complete Program Examples

Integrating all four concepts together

Program 1: Simple Calculator

Calculator.java

```
import java.util.Scanner;

public class Calculator {
        public static void main(String[] args) {
                Scanner sc = new Scanner(System.in);

                System.out.print("Enter first number  : ");
                double a = sc.nextDouble();

                System.out.print("Enter second number : ");
                double b = sc.nextDouble();

                // Arithmetic Operators
                System.out.println("Sum         = " + (a + b));
                System.out.println("Difference  = " + (a - b));
                System.out.println("Product     = " + (a * b));
                System.out.println("Quotient    = " + (a / b));
                System.out.println("Remainder   = " + (a % b));

                // Relational Operator
                System.out.println("Is a > b?   = " + (a > b));

                sc.close();
        }
}
```

Program 2: Student Grade System

GradeSystem.java

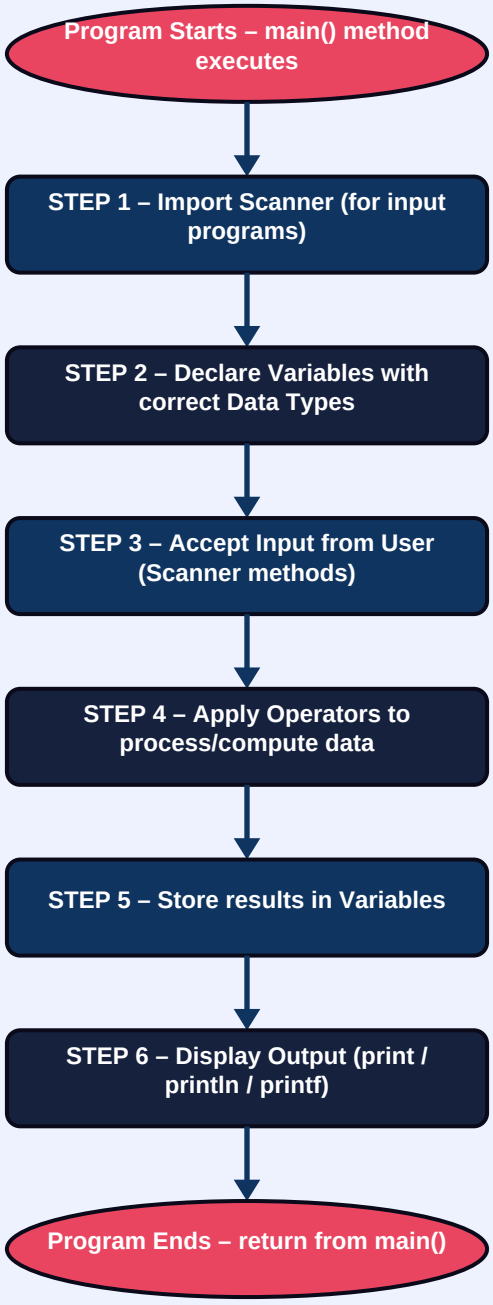
```
import java.util.Scanner;

public class GradeSystem {
        public static void main(String[] args) {
                Scanner sc = new Scanner(System.in);

                System.out.print("Enter student name  : ");
                String name = sc.nextLine();

                System.out.print("Enter marks (0-100): ");
                int marks = sc.nextInt();

                // Variables and Operators
                boolean isPassed = marks >= 40;
                char grade;

                if (marks >= 90)       grade = 'O';   // Outstanding
                else if (marks >= 75)  grade = 'A';   // First Class
                else if (marks >= 60)  grade = 'B';   // Second Class
                else if (marks >= 40)  grade = 'C';   // Pass
                else                   grade = 'F';   // Fail

                // Output
                System.out.println("\n=== RESULT CARD ===");
                System.out.println("Name   : " + name);
                System.out.println("Marks  : " + marks);
                System.out.println("Grade  : " + grade);
                System.out.println("Status : " + (isPassed ? "PASS" : "FAIL"));

                sc.close();
        }
}
```

07 Quick Reference Summary

All key concepts at a glance

Complete Java Fundamentals Flowchart

Complete Java Program Flow – Fundamentals Summary



Key Takeaways

Concept	Core Idea	Key Syntax

Variables	Named memory containers for data	`dataType varName = value;`
Data Types	Defines what value a variable holds	`int / float / double / char / boolean / String`
Operators	Symbols for math, logic, comparison	`+ - * / % == && || += ++`
Input (I/O)	Receive data from user	`Scanner sc = new Scanner(System.in);`
Output (I/O)	Display results to user	`System.out.println("...");`

■ **Pro Tip:** Every Java program follows the same pattern: **Declare variables** → **Accept input** → **Apply operators** → **Display output.** Master these four steps and you have the foundation for any Java application!